

DEVELOPMENT PLAN — Backend DriveO

Tim: 2 Backend Developer, Paralel, Minim Blocking

Versi	1.0
Tanggal	4 Oktober 2026
Basis	SRS DriveO v1.0 (97 FR, 44 BR, 22 SEC)
Tech stack	Python, FastAPI, PostgreSQL, REST API, JWT (SEC-003: access token pendek + refresh token server-side), Docker, Alembic, Pytest, Object Storage
Prinsip utama	Parallelizability > pembagian yang terlihat sama rata. Tidak ada developer yang menunggu implementasi internal developer lain — hanya contract.

1. Backend Architecture Overview

1.1 Keputusan arsitektur: Modular Monolith

Dipilih: satu aplikasi FastAPI (modular monolith) + satu database PostgreSQL + tabel yang dipisahkan per domain.

Struktur:

```
app/
├─ main.py                # FastAPI app, middleware (auth, tracing, error envelope)
├─ core/                  # config, security (JWT), db session, logging [Foundation]
├─ contracts/             # <-- KONTRAK ANTAR DOMAIN (lihat Bagian 8)
│   ├── schemas.py        # Pydantic schemas lintas domain
│   ├── ports.py          # Python Protocol (interface) per contract
│   └─ mocks.py           # implementasi mock untuk parallel dev
├─ domains/
│   ├── auth/             # A: router, service, models
│   ├── users/            # A
│   ├── rentals/          # A
│   ├── vehicles/         # A
│   ├── listings/         # A
│   ├── search/           # A
│   ├── bookings/        # A
│   ├── reviews/         # A
│   ├── payments/         # B
│   ├── ledger/           # B
│   ├── verifications/    # B (identitas penyewa)
│   ├── agreements/       # B
│   ├── handovers/        # B
│   ├── returns/          # B
│   ├── deposits/         # B
│   ├── disputes/         # B
│   ├── payouts/          # B
│   ├── notifications/    # Foundation (infra)
│   └─ audit/             # Foundation (infra)
├─ integrations/
│   ├── payment_gateway/   # B: client + webhook handler (mock & real)
│   ├── ekyc/              # B: client (mock & real)
│   ├── notification/      # Foundation: dispatcher + provider clients
│   └─ storage/            # Foundation: object storage client
└─ tests/
    ├── unit/
    ├── contract/          # mock vs real, satu suite untuk keduanya
    ├── integration/
    └─ e2e/
```

Alasan menolak microservices untuk MVP (keputusan sadar, bukan dogma):

- 1. Alur uang DriveO (DP → escrow → pelunasan → payout → refund) membutuhkan **transisi state atomik lintas booking/payment/ledger**. Dalam microservices ini menjadi distributed transaction (saga) — kompleksitas yang tidak sebanding untuk tim 2 orang.
- 2. Satu deployable = satu pipeline CI/CD, satu observability stack, satu migration history.
- 3. Batas domain tetap ditegakkan via **aturan kode** (tidak ada import model lintas domain; akses hanya via `contracts/ports.py`), bukan via batas jaringan. Jika suatu domain kelak perlu dipisah, port-nya sudah menjadi seam alami.

Aturan keras modularitas (ditegakkan di code review):

- 1. Domain A **dilarang** mengimpor model/SQLAlchemy dari domain B, dan sebaliknya. Pelanggaran = PR ditolak.
- 2. Komunikasi lintas domain hanya via **Port (Protocol)** di `app/contracts/ports.py` , di-inject via dependency injection FastAPI.
- 3. Referensi lintas tabel memakai kolom UUID **tanpa ORM relationship lintas domain**. Foreign key di level database **diperbolehkan** untuk integritas referensial (mis. `payments.booking_id → bookings.id`), karena satu database — tetapi join/query lintas domain tetap dilarang di kode aplikasi.

1.2 Pola komunikasi antar domain

Pola	Dipakai untuk	Contoh
Synchronous call via Port (in-process)	Butuh jawaban segera	Booking cek status escrow sebelum buka kunci handover
Domain event → Notification/Audit	Fire-and-forget, tidak boleh memblokir	<code>dp_paid</code> → kirim WhatsApp; semua event → audit log
Scheduler (Background Scheduler, aktor SRS)	Aksi berbasis waktu	Kedaluwarsa link DP, auto-cancel SLA, claim window 24 jam, payout H+1

Tidak ada message broker pada MVP. Event dikirim via pemanggilan dispatcher in-process (dengan outbox table untuk notifikasi agar tidak hilang saat crash). Broker (Redis/RabbitMQ) adalah optimasi nanti, bukan kebutuhan sekarang.

2. Domain Map

Pemetaan 25 domain SRS → FR → owner.

Domain	FR SRS	Owner	Keterangan
A. Foundation	—	Bersama (Sprint 0)	core/, docker, alembic, pytest, CI
B. Authentication	FR-AUTH-001-005	A	JWT, refresh, reset password
C. User	FR-USER-001-005	A	profil, consent, hapus data
D. Rental	FR-RENTAL-001-007	A	onboarding, staf, rekening payout
E. Vehicle	FR-VEHICLE-001-005	A	CRUD + foto + dokumen
F. Listing	FR-LISTING-001-005	A	harga all-in, publish, freshness
G. Search	FR-SEARCH-001-004	A	filter, sort, penanda kebaruan
H. Availability	(modul 5.5)	A	kalender per unit, kunci slot atomik
I. Booking	FR-BOOKING-001-008	A	state machine booking
J. Payment	FR-PAYMENT-001-009	B	DP, pelunasan, webhook, refund
K. Verification	FR-VERIFICATION-001-006	B	e-KYC penyewa (KTP+SIM)
L. Agreement	FR-VERIFICATION-007	B	click-to-accept + audit trail

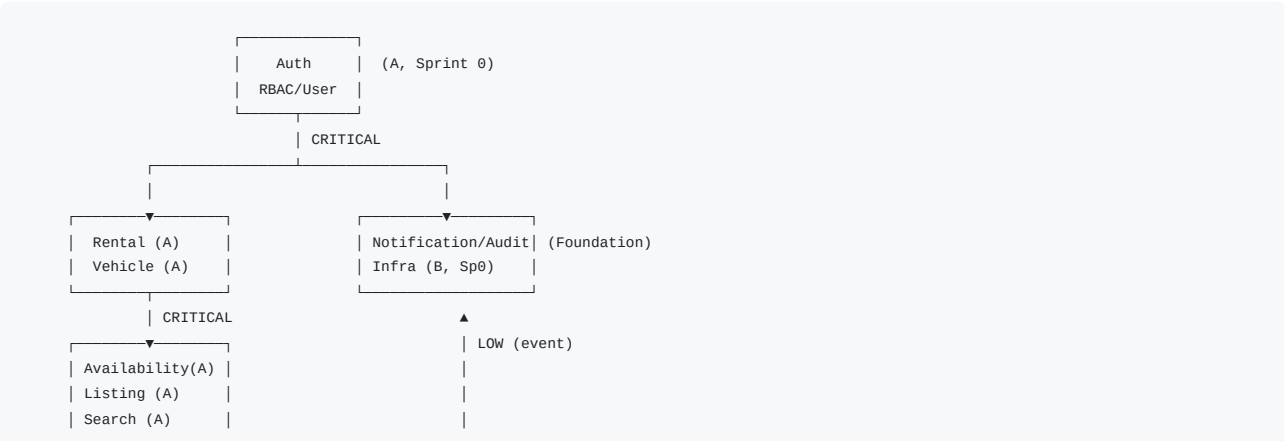
Domain	FR SRS	Owner	Keterangan
M. Handover	FR-HANDOVER-001-004	B	kunci sampai LUNAS, checklist+foto
N. Return	FR-RETURN-001-004	B	checklist, denda telat
O. Deposit	FR-DEPOSIT-001-006	B	claim window 24 jam
P. Claim	FR-DEPOSIT-003-005	B	bagian dari domain Deposit
Q. Refund	FR-PAYMENT-007	B	hanya dari dana ditahan (aturan keras)
R. Payout	FR-PAYOUT-001-005	B	H+1, retry, Dana Saya
S. Ledger	(modul 5.26)	B	immutable, sumber kebenaran dana
T. Review	FR-REVIEW-001-004	A	dua arah; eligibilitas via kontrak booking
U. Dispute	FR-DISPUTE-001-006	B	mediasi, eskalasi
V. Notification	FR-NOTIFICATION-001-004	Foundation (B lead)	dispatcher + provider
W. Admin	FR-ADMIN-001-006	Bersama	A: moderasi rental/listing; B: oversight refund/payout
X. Audit	FR-AUDIT-001-003	Foundation (B lead)	append-only
Y. External Integration	(Bagian 10 SRS)	B lead (PG, e-KYC); Foundation (storage, notif provider)	semua PROVIDER TBD → mulai dari mock

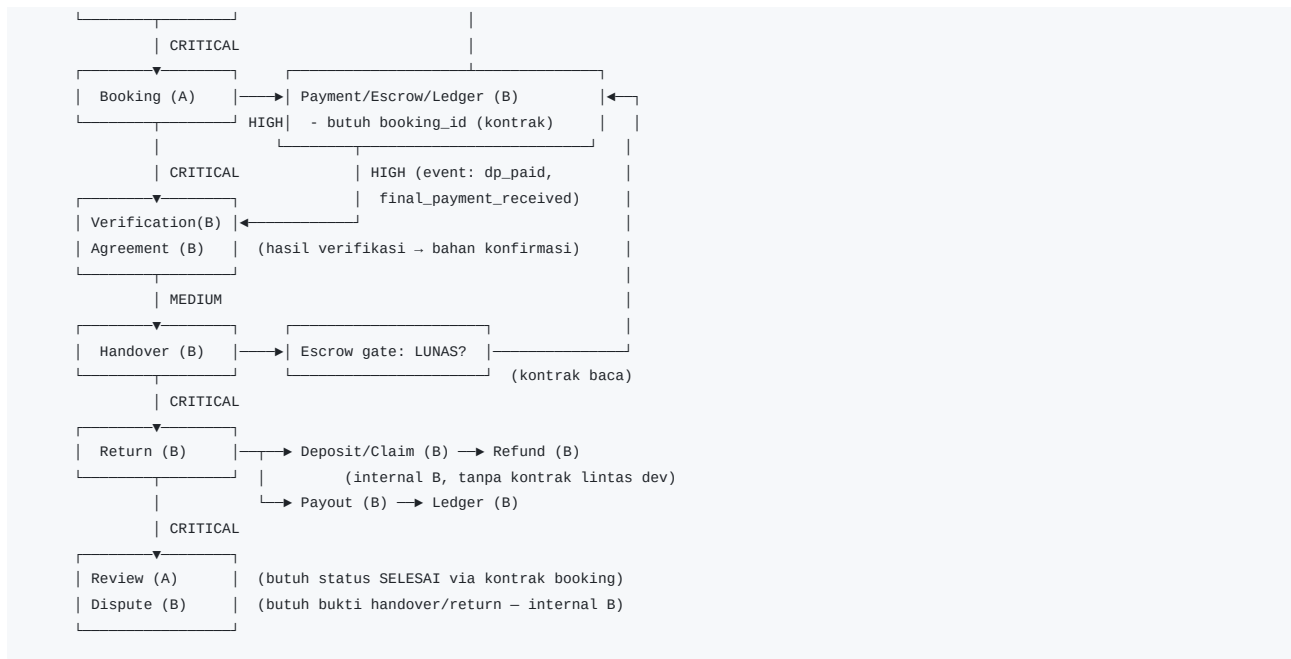
Keputusan yang disengaja:

- 1. **Review (T)** → **A**, bukan B. Alasannya: review adalah permukaan marketplace (read-heavy, tampil di listing). Aturan "hanya transaksi terselesaikan" ditegakkan A via kontrak status booking — tidak perlu B. Ini juga menyeimbangkan beban: domain B sudah paling dalam (uang).
- 2. **Claim (P) digabung ke Deposit (O)** — di SRS pun FR-nya satu kelompok (FR-DEPOSIT-003-005). Tidak perlu domain terpisah.
- 3. **Admin (W) dibagi dua**: moderasi rental/listing (A, dekat domainnya) dan oversight refund/payout (B, dekat uangnya). Keduanya memakai RBAC yang sama dari foundation.
- 4. **Notification & Audit (V, X) = foundation**, dibangun sekali di Sprint 0, dipakai semua domain via event contract. B memimpin keduanya karena notification paling kritis untuk event uang (BR-032).

3. Dependency Graph

3.1 Graph (dibaca dari atas = dibangun lebih dulu)





Catatan kritis: Booking ↔ Payment adalah **dependensi dua arah** (booking butuh status payment; payment webhook butuh mengubah booking). Ini dipecahkan BUKAN dengan "siapa dulu", melainkan dengan dua kontrak satu arah + event (lihat Bagian 8, kontrak C1 & C2). Keduanya bisa dibangun paralel penuh.

3.2 Tabel kekuatan dependensi

Dari → Ke	Kekuatan	Sifat	Cara diputus
Semua → Auth/RBAC	CRITICAL	Setiap request butuh token+role	Foundation Sprint 0; kontrak: <code>get_current_user</code> , <code>require_role</code>
Booking → Payment (status)	HIGH	Booking perlu tahu DP lunas / LUNAS	Kontrak C1 + Mock Payment
Payment → Booking (transisi)	HIGH	Webhook DP mengubah BookingState	Kontrak C2 + Mock Booking
Booking → Verification (hasil)	MEDIUM	Layar konfirmasi rental butuh ringkasan verifikasi	Kontrak C4 + Mock Verification
Handover → Escrow (gate)	CRITICAL	Checklist terkunci sampai LUNAS — aturan keras	Kontrak C3 (baca escrow) + Mock
Return → Deposit/Payout	HIGH	return_completed memicu ketiganya	Internal B (satu developer) — tanpa kontrak lintas
Listing/Search → Availability	HIGH	Ketersediaan real dari kalender	Internal A — tanpa kontrak lintas
Review → Booking (status)	MEDIUM	Eligibilitas: hanya SELESAI	Kontrak C2 (baca saja)
Semua → Notification	LOW	Fire-and-forget event	Kontrak C7 (event schema), Sprint 0
Semua → Audit	LOW	Fire-and-forget event	Kontrak C8 (event schema), Sprint 0
Payout → Rental (rekening)	MEDIUM	Butuh data rekening payout	Kontrak C9 (baca profil rental) + Mock
Dispute → Handover/Return (bukti)	MEDIUM	Butuh foto & checklist	Internal B — tanpa kontrak lintas
Admin → Semua	LOW	Read/moderate	RBAC + read API per domain

Prinsip: dependensi INTRA-developer (A → A, B → B) tidak butuh kontrak formal — cukup kesepakatan interface biasa. Kontrak formal (Bagian 8) HANYA untuk dependensi LINTAS developer.

4. Domain Ownership

Developer A — Marketplace & Rental Operations

Domain: Auth, User, Rental, Vehicle, Availability, Listing, Search, Booking, Review, Rental Dashboard, Customer History, Offline Orders.

Karakteristik domainnya: read-heavy, CRUD + state machine booking, banyak endpoint untuk frontend marketplace & dashboard rental. Kompleksitasnya di **volume dan aturan tampilan** (harga all-in, penanda kebaruan, freshness), bukan di invariansi finansial.

Yang TIDAK boleh disentuh A: tabel `payments`, `ledger_entries`, `payouts`, `refunds`, `deposits`, `claims`, `disputes`, `identity_verifications`, `handover_*`, `return_*`. Butuh data dari sana → via kontrak (Bagian 8).

Developer B — Trust, Payment & Transaction Infrastructure

Domain: Payment (+webhook PG), Escrow/Ledger, Identity Verification, Agreement, Handover, Return, Deposit/Claim, Refund, Payout, Dispute, Notification infra, Audit infra, External integrations (PG, e-KYC).

Karakteristik domainnya: write-critical, state machine ketat, invariansi uang (ledger seimbang, refund ≤ dana ditahan, idempotency). Kompleksitasnya di **ketepatan**, bukan volume. B boleh memiliki lebih sedikit domain — kedalamannya lebih besar. (Sesuai prinsip: parallelizability > kesamaan jumlah.)

Yang TIDAK boleh disentuh B: tabel `bookings`, `listings`, `vehicles`, `rentals` (kecuali baca via kontrak), `reviews`. Butuh mengubah BookingState (mis. webhook DP) → via **Kontrak C2** (BookingPort), bukan UPDATE langsung ke tabel bookings.

Foundation — dikerjakan bersama di Sprint 0

Item	Lead	Konsumen
JWT auth, RBAC middleware, <code>get_current_user</code>	A	Semua
User & consent & hapus data	A	Semua
Docker Compose, Alembic baseline, Pytest setup, CI	B	Semua
Logging terstruktur, tracing, error envelope	B	Semua
Notification dispatcher + outbox + provider client (mock)	B	Semua (via event)
Audit log append-only + query API	B	Semua (via event)
Object storage client	B	A (foto kendaraan), B (foto handover)
Kontrak C1–C9 + mock (Bagian 8)	Berdua	A & B

Cara B bekerja tanpa menunggu A di Sprint 0: B membangun payment/escrow melawan `BookingPort` mock dari hari pertama. **Cara A bekerja tanpa menunggu B:** A membangun booking melawan `PaymentPort` mock dari hari pertama. Kontrak ditulis BERDUA sebelum kode domain — itulah Sprint 0.

5. Database Ownership

5.1 Keputusan: Option A — satu database PostgreSQL, tabel dipisahkan per domain

Bukan Option B (schema per domain — menambah friksi tanpa manfaat pada skala ini; `search_path` juggling mempersulit debugging) dan **bukan** Option C (microservices — ditolak di Bagian 1.1).

5.2 Tabel per owner

Developer A owns:

Tabel	Domain
users , roles , user_consent , data_deletion_requests	Auth/User
rentals , rental_verifications , rental_staff , rental_payout_accounts	Rental
vehicles , vehicle_documents , vehicle_photos	Vehicle
vehicle_availability	Availability
listings	Listing
bookings , offline_orders	Booking
reviews	Review

Developer B owns:

Tabel	Domain
payments , payment_events , idempotency_keys	Payment
ledger_entries	Ledger
identity_verifications	Verification
agreement_acceptances , agreement_versions	Agreement
handover_checklists , handover_photos	Handover
return_checklists , return_photos	Return
deposits , claims	Deposit/Claim
refunds	Refund
payouts	Payout
disputes , dispute_evidences	Dispute

Foundation (dibuat Sprint 0, dimiliki bersama — perubahan butuh persetujuan keduanya):

Tabel	Lead
notification_outbox , notification_preferences	B
audit_logs	B

5.3 Aturan kepemilikan (kontrak database)

1. **Satu tabel = satu owner.** Tidak ada dua developer menulis migration untuk tabel yang sama tanpa persetujuan eksplisit.
2. **Referensi lintas owner** memakai kolom `UUID` (mis. `payments.booking_id`). Foreign key database **diperbolehkan** untuk integritas (dibuat oleh owner tabel anak, dengan persetujuan owner tabel induk), tetapi **dilarang** membuat ORM `relationship()` lintas domain dan **dilarang** join/query langsung ke tabel owner lain.
3. **Membaca data owner lain = via Port** (Bagian 8), bukan via query. Contoh: B butuh nama rental untuk payout → `RentalReadPort.get()`, bukan `SELECT FROM rentals`.
4. **Perubahan schema yang memengaruhi kontrak** (kolom yang dibaca lintas domain) harus diumumkan di PR dengan label `contract-change` dan disetujui kedua developer SEBELUM merge.
5. **Dilarang** `SELECT *` lintas domain dalam alasan apa pun; daftar kolom eksplisit sesuai schema kontrak.

6. API Ownership

Tabel: Domain | Endpoint | Owner | Consumer | Dependency | Mock Available? | Status awal.

"Mock Available?" = apakah consumer bisa berkembang tanpa menunggu owner selesai (via kontrak + mock di Sprint 0).

Domain	Endpoint	Owner	Consumer	Dependency	Mock?	Status
Auth	/auth/*	A	Semua	—	Ya (Sprint 0)	Sprint 0
Users	/users/*	A	Semua	Auth	Ya	Sprint 0
Rentals	/rentals/*	A	Frontend, B (payout)	Auth, User	Ya	Sprint 1
Vehicles	/rentals/{id}/vehicles/*	A	Frontend	Rental	— (internal A)	Sprint 1
Availability	/vehicles/{id}/availability	A	Listing, Booking	Vehicle	— (internal A)	Sprint 1
Listings	/listings/*	A	Frontend	Vehicle, Availability	— (internal A)	Sprint 2
Search	/search	A	Frontend	Listing	— (internal A)	Sprint 2
Bookings	/bookings/*	A	Frontend, B	Payment (C1), Verification (C4)	Ya (C1, C4)	Sprint 2
Payments	/payments/*	B	A, Frontend	Booking (C2)	Ya (C2)	Sprint 1
Webhook PG	/webhooks/payment-gateway	B	PG (eksternal)	Booking (C2)	Ya (C2 + fixture)	Sprint 1
Verifications	/verifications/*	B	A, Frontend	Booking (C2)	Ya (C2)	Sprint 2
Agreements	/agreements/*	B	A (precondition)	—	Ya	Sprint 2
Handovers	/bookings/{id}/handover/*	B	Frontend	Booking (C2), Escrow (C3)	Ya (C2, C3)	Sprint 3
Returns	/bookings/{id}/return/*	B	Frontend	Booking (C2)	Ya	Sprint 3
Deposits/Claims	/deposits/*, /claims/*	B	Frontend	Return (internal B)	— (internal B)	Sprint 3
Refunds	/refunds/*	B	A (cancel flow)	Payment/Ledger (internal B)	Ya (C5)	Sprint 2
Payouts	/payouts/*, /rentals/{id}/funds	B	A (Dana Saya), Frontend	Return, Rental (C9)	Ya (C9)	Sprint 4

Domain	Endpoint	Owner	Consumer	Dependency	Mock?	Status
Reviews	<code>/bookings/{id}/reviews</code> , <code>/listings/{id}/reviews</code>	A	Frontend	Booking (C2, internal A)	— (internal A)	Sprint 4
Disputes	<code>/disputes/*</code>	B	Frontend, CS	Handover/Return (internal B)	— (internal B)	Sprint 4
Notifications	<code>/notifications/*</code> (inbox, preferensi)	Foundation/B	Semua	Event (C7)	Ya	Sprint 0
Admin	<code>/admin/*</code>	A+B	Internal	RBAC	—	Sprint 4

Kontrak C1-C9 didefinisikan di Bagian 8. "Internal" = satu developer, tidak butuh kontrak formal.

7. Shared Infrastructure Ownership

Semua di bawah ini dibangun di **Sprint 0** dan dimiliki bersama setelahnya (perubahan butuh persetujuan keduanya).

Komponen	Lead	Isi
Docker Compose	B	<code>postgres</code> , <code>api</code> , (opsional <code>mailhog</code> untuk email dev). Satu perintah: <code>docker compose up</code> → API + DB siap.
Alembic baseline	B	<code>alembic init</code> , <code>env.py</code> (async engine), migration <code>0001_baseline</code> (tabel foundation). Aturan: Bagian 15.
Pytest setup	B	<code>pytest.ini</code> , fixture DB (test database terpisah), factory dasar, marker <code>unit/contract/integration/e2e</code> .
CI (GitHub Actions)	B	Job: lint (ruff) → typecheck (mypy) → pytest → migration check (<code>upgrade head</code> + <code>downgrade -1</code> + <code>upgrade head</code>).
Logging & tracing	B	JSON log, <code>trace_id</code> per request (wajib untuk alur uang, SRS 15.2), korelasi dengan audit.
Error envelope	B	Format error baku: <code>{ "error": { "code": "BOOKING_SLOT_CONFLICT", "message": "...", "trace_id": "..." } }</code> . Kode error per domain didaftarkan di <code>app/core/errors.py</code> .
Pagination & filter	A	Skema baku <code>?page=&size=&sort=</code> + response envelope <code>{ "data": [...], "meta": {...} }</code> .
JWT & RBAC	A	<code>get_current_user</code> , <code>require_roles(...)</code> sebagai FastAPI dependencies; sesuai SEC-003 (access token pendek + refresh token server-side).
Password hashing	A	<code>argon2id</code> (keputusan teknis; SRS membolehkan — SEC-005 menandai pilihan algoritma sebagai technical decision).
Object storage client	B	Interface <code>StoragePort</code> : <code>upload()</code> , <code>signed_url()</code> ; implementasi lokal (filesystem) untuk dev + S3-compatible untuk prod.
Notification dispatcher	B	<code>NotificationPort.publish(event)</code> → tulis ke <code>notification_outbox</code> → worker kirim via provider (mock di Sprint 0-3).
Audit writer	B	<code>AuditPort.record(event)</code> → append-only ke <code>audit_logs</code> ; dipanggil via dependency, tidak memblokir request (best-effort dengan retry).

8. Contract Matrix

Definisi contract: Python `Protocol` di `app/contracts/ports.py` + Pydantic schema di `app/contracts/schemas.py` + implementasi mock di `app/contracts/mocks.py`. Kontrak adalah **hukum**: siapa pun boleh mengubah implementasi di pihaknya, tetapi tidak boleh mengubah signature kontrak tanpa persetujuan kedua developer.

C1 — PaymentStatus (B → A)

Dibutuhkan A (booking) untuk: menampilkan status bayar, memutuskan link DP kedaluwarsa, dan (via C3) membuka kunci handover.

```
class PaymentStatus(str, Enum):
    MENUNGGU = "MENUNGGU"          # link terbit, belum dibayar
    DP_DITERIMA = "DP_DITERIMA"     # event dp_paid (bukan state, lihat SRS 7)
    LUNAS = "LUNAS"                 # total 100% diterima di escrow
    KEDALUWARSA = "KEDALUWARSA"
    GAGAL = "GAGAL"

class PaymentReadPort(Protocol):
    def get_by_booking(self, booking_id: UUID) -> PaymentSummary: ...
    # PaymentSummary { booking_id, status: PaymentStatus,
    #   dp_amount: int, paid_amount: int, remaining_amount: int, currency: "IDR" }

class EscrowReadPort(Protocol):          # C3
    def get_state(self, booking_id: UUID) -> EscrowState: ...
    # EscrowState: MENUNGGU_DANA | DITAHAN_ESCROW | LUNAS | DITERUSKAN
    #               | DIREFUND_SEBAGIAN | DIREFUND_PENUH | DIBEKUKAN_KLAIM
```

Mock: `MockPaymentReadPort` dengan state yang bisa di-set manual per test (`mock.set_status(booking_id, LUNAS)`).

C2 — BookingState (A → B)

Dibutuhkan B untuk: webhook payment mengubah BookingState; handover/return validasi status booking; review eligibilitas (dipakai A sendiri juga).

```
class BookingState(str, Enum):
    MENUNGGU_DP = "MENUNGGU_DP"
    MENUNGGU_KONFIRMASI_RENTAL = "MENUNGGU_KONFIRMASI_RENTAL"
    TERKONFIRMASI = "TERKONFIRMASI"
    MENUNGGU_PELUNASAN = "MENUNGGU_PELUNASAN"
    DALAM_SEWA = "DALAM_SEWA"
    SELESAI = "SELESAI"
    DIBATALKAN = "DIBATALKAN"
    KEDALUWARSA = "KEDALUWARSA"
    DITOLAK_RENTAL = "DITOLAK_RENTAL"

class BookingPort(Protocol):
    def get(self, booking_id: UUID) -> BookingSummary: ...
    # BookingSummary { booking_id, rental_id, vehicle_id, penyewa_id,
    #   state: BookingState, tanggal_mulai, tanggal_selesai,
    #   total_amount: int, agreement_accepted: bool }
    def transition(self, booking_id: UUID, to_state: BookingState,
                  reason: str, actor: str) -> BookingSummary: ...
    # transition HANYA untuk transisi yang diizinkan state machine (SRS 7.1);
    # transisi terlarang -> raise BookingTransitionForbidden (403/409).
```

Aturan penting: B tidak pernah `UPDATE bookings` langsung. Semua perubahan via `BookingPort.transition()` — yang di dalamnya A menegakkan state machine + audit.

C4 — VerificationResult (B → A)

Dibutuhkan A untuk layar konfirmasi rental (BR-015: hasil verifikasi sebagai bahan konfirmasi).

```
class VerificationResult(str, Enum):
    DISETUJUI = "DISETUJUI"; DITOLAK = "DITOLAK"
    MENUNGGU = "MENUNGGU"; BUTUH_REVIEW_MANUAL = "BUTUH_REVIEW_MANUAL"

class VerificationReadPort(Protocol):
    def get_by_booking(self, booking_id: UUID) -> VerificationSummary | None: ...
```

```
# VerificationSummary { booking_id, result: VerificationResult,
#   verified_at: datetime | None, rejection_reason: str | None }
# CATATAN: tidak ada citra dokumen – rental tidak boleh melihat KTP mentah (SEC-006).
```

C5 — RefundRequest (A → B)

Dibutuhkan A untuk alur pembatalan (FR-BOOKING-004/005/008): A menghitung *aturan* refund (tier — TBD), B mengeksekusi *pergerakan dana*.

```
class RefundPort(Protocol):
    def request_refund(self, booking_id: UUID, amount: int,
        reason: str, requested_by: str) -> RefundReceipt: ...
    # RefundReceipt { refund_id, booking_id, amount, status }
    # ATURAN KERAS: B menolak bila amount > dana yang ditahan (BR-010).
```

C7 — NotificationEvent (semua → Foundation)

```
class NotificationEvent(BaseModel):
    event_id: UUID
    type: str # mis. "dp_paid", "payout_completed", "booking_confirmed"
    booking_id: UUID | None
    recipient_role: str # "penyewa" | "rental" | ...
    recipient_id: UUID
    payload: dict # data untuk template pesan
    idempotency_key: str # = event_id; pengiriman ganda dicegah

class NotificationPort(Protocol):
    def publish(self, event: NotificationEvent) -> None: ...
```

C8 — AuditEvent (semua → Foundation)

```
class AuditEvent(BaseModel):
    actor: str; actor_id: UUID | None; action: str
    entity: str; entity_id: UUID | None
    before: dict | None; after: dict | None
    reference: str | None # mis. nomor referensi PG/bank
    trace_id: str
```

C9 — RentalRead (A → B)

Dibutuhkan B untuk payout (rekening tujuan) dan notifikasi ke rental.

```
class RentalReadPort(Protocol):
    def get(self, rental_id: UUID) -> RentalSummary: ...
    # RentalSummary { rental_id, nama, payout_account_masked: str,
    #   status_verifikasi: str }
    # CATATAN: nomor rekening penuh TIDAK diekspos; B membaca dari
    # tabelnya sendiri? TIDAK – rekening payout disimpan di tabel A
    # (rental_payout_accounts). B mengakses via port ini; nilai penuh
    # hanya dibuka di dalam service payout B, tidak pernah di-log.
```

Tabel ringkasan kontrak

ID	Kontrak	Owner (implementasi)	Konsumen	Mock di Sprint 0?
C1	PaymentStatus + C3 EscrowState	B	A	Ya
C2	BookingState (baca + transisi)	A	B	Ya
C4	VerificationResult	B	A	Ya

ID	Kontrak	Owner (implementasi)	Konsumen	Mock di Sprint 0?
C5	RefundRequest	B	A	Ya
C7	NotificationEvent	Foundation (B)	A, B	Ya (log)
C8	AuditEvent	Foundation (B)	A, B	Ya (log)
C9	RentalRead	A	B	Ya

Setiap kontrak memiliki **contract test** (`tests/contract/test_<kontrak>.py`) yang dijalankan terhadap implementasi mock DAN implementasi real — suite yang sama, dua implementasi. Jika mock lolos tapi real gagal, kontraknya yang salah, bukan mock-nya.

9. Mock Strategy

9.1 Prinsip

1. Mock **mengimplementasikan Port yang sama** dengan production — tidak ada kontrak ganda.
2. Mock bersifat **deterministik dan dapat di-set** dari test (`mock.set_*`), bukan random.
3. Pemilihan mock vs real via **dependency injection** (bukan `if` berserakan): `settings.USE MOCKS` atau override dependency di test.
4. Mock **dihapus dari path production** secara bertahap per checkpoint (Bagian 13) — bukan sekaligus di akhir.

9.2 Daftar mock

Mock	Mensimulasikan	Bentuk	Dipakai oleh
MockPaymentGateway	PG eksternal: buat link bayar, callback sukses/gagal/kedaluwarsa	Fake service + fixture JSON webhook; callback dipicu manual via endpoint <code>/__mock__/_pg/callback</code> (hanya di env dev/test)	B (Sprint 1), A (Sprint 2, via C1)
MockBookingPort	BookingPort milik A	In-memory dict <code>booking_id</code> → <code>BookingSummary</code> ; <code>transition()</code> menegakkan subset state machine	B (Sprint 1–2)
MockPaymentReadPort / MockEscrowReadPort	C1/C3 milik B	In-memory; <code>set_status()</code> per test	A (Sprint 2–3)
MockVerificationPort	C4 milik B	Kembalikan <code>DISETUJUI / DITOLAK</code> sesuai setup	A (Sprint 2)
MockRentalReadPort	C9 milik A	Data rental fiktif + rekening masked	B (Sprint 4)
MockEKYC	Vendor e-KYC: OCR KTP+SIM	Kembalikan hasil OCR canned (lolos / meragukan / palsu) berdasar fixture file	B (Sprint 2)
MockNotificationProvider	WA/push/email	Tulis ke <code>notification_outbox</code> + log; test assert dari outbox, bukan dari provider	Semua (Sprint 0–3)
MockStorage	Object storage	Filesystem lokal; <code>signed_url()</code> = URL lokal bertanda waktu	A (Sprint 1), B (Sprint 3)

9.3 Contoh pola DI (FastAPI)

```
# app/contracts/ports.py
def get_payment_port() -> PaymentReadPort:
    if settings.USE MOCKS:
        return MockPaymentReadPort()
    return RealPaymentReadPort(db_session=...)
```

```
# app/domains/bookings/service.py
class BookingService:
    def __init__(self, payment_port: PaymentReadPort = Depends(get_payment_port)):
        ...
```

Test contract (sama untuk mock & real):

```
@pytest.mark.parametrize("port", [MockPaymentReadPort(), RealPaymentReadPort(test_db)])
def test_payment_status_contract(port):
    summary = port.get_by_booking(BOOKING_ID)
    assert summary.currency == "IDR"
    assert summary.paid_amount + summary.remaining_amount == summary.dp_amount # invariant
```

9.4 Yang TIDAK di-mock

- **State machine** (booking & escrow): selalu implementasi asli, bahkan di test — karena di sinilah bug paling mahal.
- **Aturan uang** (refund ≤ dana ditahan, ledger seimbang): tidak ada mock yang melonggarkan ini.
- **Auth/RBAC**: selalu real (JWT asli) di semua test integration/e2e.

10. Developer A Work Breakdown — Marketplace & Rental Operations

Estimasi dalam hari kerja (1 dev). Status awal semua: `Not Started`.

A-001 — Authentication (JWT)

- **Domain:** Auth (FR-AUTH-001-005) · **Input:** — · **Output:** `get_current_user`, `require_roles`, token service
- **Dependency:** Tidak ada · **Can Start When:** Sprint 0 hari pertama
- **Deliverable:** registrasi/login/logout/refresh/reset-password; JWT access pendek + refresh server-side (SEC-003); hashing argon2id
- **Acceptance:** login salah → 401 generik (tanpa user enumeration); refresh reuse → seluruh sesi dicabut; semua endpoint sensitif menolak tanpa token · **Estimasi:** 3 hari

A-002 — RBAC & Authorization Enforcement

- **Domain:** Auth (SEC-001/002, Bagian 12 SRS) · **Input:** A-001 · **Output:** middleware/decorator otorisasi per (role, resource, aksi)
- **Dependency:** A-001 · **Can Start When:** A-001 selesai
- **Deliverable:** matriks otorisasi sebagai kode (bukan dokumen mati); segregasi tugas finansial
- **Acceptance:** 22 kapabilitas × 6 peran diuji (authorization test SRS 16.4); upaya IDOR → 403 + audit · **Estimasi:** 2 hari

A-003 — User Profile, Consent & Data Deletion

- **Domain:** User (FR-USER-001-005) · **Input:** A-001 · **Output:** `/users/*`
- **Dependency:** A-001 · **Can Start When:** A-001 selesai
- **Deliverable:** profil, consent eksplisit terpisah (BR-040), pengajuan hapus data (BR-016/SEC-021 — dengan penolakan bila sengketa aktif)
- **Acceptance:** hapus data dengan sengketa aktif → ditolak sebagian + alasan tertulis · **Estimasi:** 2 hari

A-004 — Rental Onboarding & Management

- **Domain:** Rental (FR-RENTAL-001, 003-007) · **Input:** A-001-003 · **Output:** `/rentals/*`
- **Dependency:** Auth/User · **Can Start When:** Sprint 1 (kontrak auth tersedia)
- **Deliverable:** registrasi rental, profil, staf multi-user, persetujuan perjanjian merchant, pengaturan rekening payout (terenkripsi, SEC-007)
- **Acceptance:** rental belum verifikasi → tidak bisa publish listing (BR-036); rekening tersimpan terenkripsi · **Estimasi:** 3 hari

A-005 — Rental Verification Workflow

- **Domain:** Rental (FR-RENTAL-002, FR-ADMIN-002) · **Input:** A-004 · **Output:** alur submit → review admin → approve/reject
- **Dependency:** A-004 · **Can Start When:** A-004 selesai
- **Deliverable:** upload dokumen (NIB, identitas PJ, dokumen kendaraan) via storage client; antrean review; badge terverifikasi
- **Acceptance:** dokumen palsu → tolak/blokir (BR-033); audit tiap keputusan · **Estimasi:** 2 hari

A-006 — Vehicle Management

- **Domain:** Vehicle (FR-VEHICLE-001-005) · **Input:** A-004 · **Output:** `/rentals/{id}/vehicles/*`
- **Dependency:** A-004 · **Can Start When:** A-004 selesai
- **Deliverable:** CRUD kendaraan, foto (validasi file SEC-018), dokumen kendaraan, nonaktifkan
- **Acceptance:** upload .exe/.php → ditolak; foto tersimpan via StoragePort (mock OK) · **Estimasi:** 2.5 hari

A-007 — Availability Calendar & Atomic Slot Lock

- **Domain:** Availability (modul 5.5, BR-028) · **Input:** A-006 · **Output:** `/vehicles/{id}/availability`, fungsi `lock_slot()` atomik
- **Dependency:** A-006 · **Can Start When:** A-006 selesai
- **Deliverable:** kalender per unit; penguncian slot dalam transaksi DB (anti double booking platform)
- **Acceptance:** dua booking tumpang tindih bersamaan → satu 409; batas jujur (order WA paralel) didokumentasikan · **Estimasi:** 2.5 hari

A-008 — Listing Management

- **Domain:** Listing (FR-LISTING-001-005) · **Input:** A-006, A-007 · **Output:** `/listings/*`
- **Dependency:** A-006, A-007 · **Can Start When:** keduanya selesai
- **Deliverable:** buat dari kendaraan, harga all-in (BR-026), publish/unpublish, freshness: penanda "diperbarui X lalu" (BR-025) + penalti peringkat listing basi (BR-027, ambang TBD)
- **Acceptance:** listing tanpa harga all-in → 422; listing basi otomatis berlabel · **Estimasi:** 2.5 hari

A-009 — Search, Filter & Sort

- **Domain:** Search (FR-SEARCH-001-004) · **Input:** A-008 · **Output:** `GET /search`
- **Dependency:** A-008 · **Can Start When:** A-008 selesai (bisa paralel dengan data fixture)
- **Deliverable:** filter lokasi/tanggal/jenis/harga + ketersediaan real + sortir; tanpa login bisa search
- **Acceptance:** tanggal selesai < mulai → 422 dengan pesan jelas; hasil kosong → saran pelonggaran · **Estimasi:** 2.5 hari

A-010 — Comparison Support

- **Domain:** Search (modul 5.8) · **Input:** A-009 · **Output:** `GET /listings/compare?ids=...`
- **Dependency:** A-009 · **Can Start When:** A-009 selesai
- **Deliverable:** perbandingan side-by-side: harga all-in, spesifikasi, profil rental, rating
- **Acceptance:** listing tidak tersedia di tengah jalan → dikeluarkan + penjelasan · **Estimasi:** 1 hari

A-011 — Booking Creation (Contract-First terhadap Payment)

- **Domain:** Booking (FR-BOOKING-001-003, 007) · **Input:** A-008, A-009 · **Output:** `POST /bookings`, penerbitan link DP, scheduler kedaluwarsa
- **Dependency:** Kontrak C1 (PaymentReadPort mock), C7/C8 · **Can Start When:** Sprint 2 — **tanpa menunggu B selesai** (pakai MockPaymentReadPort)
- **Deliverable:** buat booking + kunci slot atomik; precondition `agreement_accepted` (422 bila belum); hitung rincian transparan (DP = TBD → tampil sebagai "belum final" bila persen belum dikonfigurasi); link DP + timer kedaluwarsa (X menit TBD → parameter)
- **Acceptance:** tanpa persetujuan perjanjian → 422; slot bentrok → 409; link kedaluwarsa → booking KEDALUWARSA + slot terbuka (diuji dengan mock payment) · **Estimasi:** 3.5 hari

A-012 — Booking Lifecycle: Confirmation & Cancellation

- **Domain:** Booking (FR-BOOKING-004–006, 008) · **Input:** A-011 · **Output:** confirm/reject/cancel endpoints + scheduler auto-cancel
- **Dependency:** Kontrak C1 (mock), C4 (MockVerificationPort), C5 (MockRefundPort) · **Can Start When:** A-011 selesai
- **Deliverable:** konfirmasi/tolak rental (dengan rincian dana transparan BR-030); auto-cancel tanpa konfirmasi dalam SLA TBD → refund penuh via C5; batal pengguna (tier TBD → parameter); batal rental (refund penuh + penalti reputasi + realokasi)
- **Acceptance:** SLA lewat → batal otomatis + DP kembali penuh (terhadap mock refund); tidak ada booking menggantung di MENUNGGU_KONFIRMASI_RENTAL · **Estimasi:** 3 hari

A-013 — Booking Extension

- **Domain:** Booking (FR-BOOKING-007) · **Input:** A-011 · **Output:** `POST /bookings/{id}/extend`
- **Dependency:** A-011, C1 (mock) · **Can Start When:** A-011 selesai
- **Deliverable:** pengajuan → persetujuan rental → pembayaran tambahan on-platform → slot tambahan dikunci
- **Acceptance:** slot tambahan bentrok → tolak langsung; bayar gagal → periode semula tetap · **Estimasi:** 1.5 hari

A-014 — Rental Dashboard & Customer History

- **Domain:** Rental ops (modul 5.3, BR-029) · **Input:** A-011 · **Output:** `/rentals/{id}/dashboard`, rekap harian, riwayat pelanggan
- **Dependency:** A-011 · **Can Start When:** A-011 selesai
- **Deliverable:** semua booking (termasuk `offline_orders` — pencatatan order non-platform), template balasan, rekap harian
- **Acceptance:** order WA yang dicatat manual muncul di rekap + mengunci kalender · **Estimasi:** 3 hari

A-015 — Offline Order Recording (Anti-Disintermediasi)

- **Domain:** Booking/Rental (BR-035) · **Input:** A-007, A-014 · **Output:** `POST /rentals/{id}/offline-orders`
- **Dependency:** A-007 · **Can Start When:** A-007 selesai
- **Deliverable:** catat order non-platform → masuk GMV + kunci slot; komisi ditagih via saldo (kolaborasi dengan B via kontrak saldo — didefinisikan saat dibutuhkan, TBD)
- **Acceptance:** order tercatat → slot terkunci; GMV mencakup nominalnya · **Estimasi:** 1.5 hari

A-016 — Review & Rating

- **Domain:** Review (FR-REVIEW-001–004) · **Input:** A-011 · **Output:** `/bookings/{id}/reviews`, tampil di listing/profil
- **Dependency:** Kontrak C2 (baca BookingState — internal A, via service sendiri) · **Can Start When:** A-011 selesai
- **Deliverable:** submit dua arah; eligibilitas = booking SELESAI (anti fake review BR-013); satu review per arah
- **Acceptance:** review untuk booking batal → 422; duplikat arah → 422 · **Estimasi:** 2 hari

A-017 — Rental Finance Read (Riwayat Dana & Dana Saya)

- **Domain:** Rental ops (BR-031/032) · **Input:** A-014 · **Output:** `GET /rentals/{id}/funds` (baca via kontrak B)
- **Dependency:** Kontrak payout/ledger read milik B (didefinisikan Sprint 3, mock tersedia) · **Can Start When:** Sprint 4 — **tanpa menunggu B selesai** (mock)
- **Deliverable:** tab Riwayat Dana per booking (immutable, timestamp + ref) + layar Dana Saya (ditahan → dalam perjalanan → ditransfer) + rekap bulanan unduhan
- **Acceptance:** setiap event uang tampil ≤ 1 menit setelah terjadi (dengan mock); format transparansi "Anda terima RpX dari RpY — komisi Z%" · **Estimasi:** 2.5 hari

A-018 — Admin: Moderasi Rental & Listing

- **Domain:** Admin (FR-ADMIN-002–003) · **Input:** A-005, A-008 · **Output:** `/admin/*` (moderasi)
- **Dependency:** A-002 (RBAC) · **Can Start When:** A-005 selesai
- **Deliverable:** antrean verifikasi rental, moderasi listing, suspend user
- **Acceptance:** hanya role Admin; semua aksi tercatat di audit · **Estimasi:** 2.5 hari

11. Developer B Work Breakdown — Trust, Payment & Transaction Infrastructure

Estimasi dalam hari kerja (1 dev). Status awal semua: `Not Started`.

B-001 — DevOps Bootstrap

- **Domain:** Foundation · **Input:** — · **Output:** docker-compose, alembic baseline, pytest, CI
- **Dependency:** Tidak ada · **Can Start When:** Sprint 0 hari pertama
- **Deliverable:** `docker compose up` → API + Postgres siap; `alembic upgrade head` hijau; CI: `ruff` → `mypy` → `pytest` → migration check
- **Acceptance:** dev baru clone → jalan < 15 menit mengikuti README · **Estimasi:** 3 hari

B-002 — Notification Infrastructure

- **Domain:** Notification (FR-NOTIFICATION-001-004, BR-032) · **Input:** B-001 · **Output:** `NotificationPort`, `notification_outbox`, `dispatcher` + `MockProvider`
- **Dependency:** B-001 · **Can Start When:** B-001 selesai
- **Deliverable:** publish event → outbox → worker kirim (mock: tulis log); preferensi & inbox in-app; idempotency per event
- **Acceptance:** event uang → notifikasi tercatat di outbox < 1 detik; kirim ganda → 1 pesan (idempoten) · **Estimasi:** 2 hari

B-003 — Audit Infrastructure

- **Domain:** Audit (FR-AUDIT-001-003, SEC-009) · **Input:** B-001 · **Output:** `AuditPort`, tabel `audit_logs` append-only
- **Dependency:** B-001 · **Can Start When:** B-001 selesai (paralel dengan B-002)
- **Deliverable:** record event (actor, before → after, trace_id); query API untuk Admin; upaya update/delete → ditolak di DB (trigger/rule)
- **Acceptance:** 24 event SRS 15.1 semuanya bisa dicatat; audit write gagal untuk aksi finansial → aksi dibatalkan (fail closed) · **Estimasi:** 1.5 hari

B-004 — Payment Initiation (DP / Full / Final)

- **Domain:** Payment (FR-PAYMENT-001-003, 009) · **Input:** B-001 · **Output:** `/payments/*`, PG client (mock), payment records
- **Dependency:** Kontrak C2 (MockBookingPort) · **Can Start When:** Sprint 1 — **tanpa menunggu A selesai**
- **Deliverable:** buat payment DP/pelunasan/full/cash-record; idempotency key per (booking, jenis); PG mock dengan endpoint callback manual
- **Acceptance:** dua inisiasi dengan key sama → 1 record (tanpa double charge); nominal < sisa → 422 · **Estimasi:** 3 hari

B-005 — Payment Webhook & Idempotency

- **Domain:** Payment (FR-PAYMENT-004-006, SEC-011/012) · **Input:** B-004 · **Output:** `POST /webhooks/payment-gateway`
- **Dependency:** B-004, Kontrak C2 (mock) · **Can Start When:** B-004 selesai
- **Deliverable:** verifikasi HMAC signature; cek duplikat per event ID; transisi payment atomik; panggil `BookingPort.transition()` (mock) untuk `dp_paid`; penanganan webhook-terlambat-setelah-kedaluwarsa → refund (aturan detail TBD)
- **Acceptance:** webhook 3× → 1 efek; signature salah → 4xx tanpa perubahan state; seluruh skenario SRS 16.6 lolos · **Estimasi:** 3 hari

B-006 — Escrow State Machine & Ledger

- **Domain:** Payment/Ledger (modul 5.14/5.26, BR-004/010/031) · **Input:** B-004, B-005 · **Output:** EscrowState service, `ledger_entries` immutable
- **Dependency:** B-005 · **Can Start When:** B-005 selesai
- **Deliverable:** `MENUNGGU_DANA` → `DITAHAN_ESCROW` → `LUNAS` → `DITERUSKAN` (+ cabang refund/beku); setiap perpindahan dana = 1 ledger entry (debit=kredit per booking)
- **Acceptance:** upaya `DITERUSKAN` sebelum `LUNAS` → ditolak; ledger selalu seimbang (test invarian); transisi terlarang → exception · **Estimasi:** 3 hari

B-007 — Refund Engine

- **Domain:** Refund (FR-PAYMENT-007, BR-010/011) · **Input:** B-006 · **Output:** RefundPort (C5) + /refunds/*
- **Dependency:** B-006 · **Can Start When:** B-006 selesai
- **Deliverable:** eksekusi refund dari dana ditahan; **aturan keras:** amount > dana tertahan → ditolak, tidak bisa di-override; tier bertingkat sebagai parameter (TBD)
- **Acceptance:** refund melebihi dana → 422; tidak ada path kode yang menulis ke "kantong operasional" (tidak ada tabelnya) · **Estimasi:** 2.5 hari

B-008 — Identity Verification (e-KYC)

- **Domain:** Verification (FR-VERIFICATION-001–006) · **Input:** B-001 · **Output:** /verifications/*, e-KYC client (mock)
- **Dependency:** Kontrak C2 (mock, untuk kait booking) · **Can Start When:** Sprint 2
- **Deliverable:** submit KTP+SIM + consent terpisah (BR-040); mock e-KYC (lolos/meragukan/palsu); antrean review manual; hasil via C4 (ringkasan saja — tanpa citra, SEC-006); retensi terjadwal (90 hari TBD → parameter)
- **Acceptance:** dokumen palsu → DITOLAK + flag; rental hanya terima ringkasan (test: tidak ada endpoint yang mengembalikan citra ke role rental) · **Estimasi:** 3 hari

B-009 — Electronic Agreement Acceptance

- **Domain:** Agreement (FR-VERIFICATION-007, BR-017) · **Input:** B-008 · **Output:** /agreements/*
- **Dependency:** — (kecil, bisa paralel B-008) · **Can Start When:** Sprint 2
- **Deliverable:** versioning dokumen; click-to-accept + audit trail (siapa, kapan, versi, hash); agreement_accepted diekspos untuk precondition booking
- **Acceptance:** versi berubah sebelum accept → accept lama tidak sah · **Estimasi:** 1.5 hari

B-010 — Handover Gate & Checklist

- **Domain:** Handover (FR-HANDOVER-001–004, BR-007) · **Input:** B-006 · **Output:** /bookings/{id}/handover/*
- **Dependency:** Kontrak C2 + C3 (mock) · **Can Start When:** Sprint 3 — tanpa menunggu A selesai
- **Deliverable:** **hard gate:** checklist TERKUNCI sampai escrow = LUNAS (ditolak di API, bukan sekadar UI); checklist + foto kedua pihak (body, odometer/BBM, kelengkapan); konfirmasi ganda → event handover_completed
- **Acceptance:** POST checklist sebelum LUNAS → 423/403 (test SRS 16.3); urutan state D6 ditegakkan · **Estimasi:** 2.5 hari

B-011 — Return & Late Fee

- **Domain:** Return (FR-RETURN-001–004) · **Input:** B-010 · **Output:** /bookings/{id}/return/*
- **Dependency:** B-010, Kontrak C2 (mock) · **Can Start When:** B-010 selesai
- **Deliverable:** checklist + foto pembanding; konfirmasi; denda per jam sesuai perjanjian (BR-020); return_completed memicu 3 hal: claim window 24 jam + payout TERJADWAL H+1 + hitung komisi
- **Acceptance:** return tanpa handover selesai → 409; keterlambatan → denda terhitung & tercatat · **Estimasi:** 3 hari

B-012 — Deposit: Claim Window, Claim & Freeze

- **Domain:** Deposit (FR-DEPOSIT-001–006, BR-009/019) · **Input:** B-011 · **Output:** /deposits/*, /claims/*, scheduler 24 jam
- **Dependency:** B-011 (internal B) · **Can Start When:** B-011 selesai
- **Deliverable:** tahan 24 jam otomatis → lepas bila tanpa klaim; klaim + bukti → DIBEKUKAN_KLAIM; putusan mediasi → potong sebagian / lepas penuh
- **Acceptance:** klaim lewat 24 jam → ditolak otomatis; seluruh perpindahan tercatat di ledger · **Estimasi:** 2.5 hari

B-013 — Dispute & Mediation

- **Domain:** Dispute (FR-DISPUTE-001–006) · **Input:** B-010, B-011, B-012 · **Output:** /disputes/*
- **Dependency:** B-010–012 (internal B) · **Can Start When:** Sprint 4

- **Deliverable:** buka sengketa per kategori SRS; lampiran bukti; alur mediasi; aksi resolusi (potong/lepas/refund); eskalasi hukum → paket dokumentasi (BR-024); notifikasi alasan tertulis
- **Acceptance:** dana terkait otomatis dibekukan saat dispute dibuka; tiap kategori SRS 3.16 punya e2e test • **Estimasi:** 3 hari

B-014 — Payout H+1 & Retry

- **Domain:** Payout (FR-PAYOUT-001–005, BR-008/030) • **Input:** B-011 • **Output:** scheduler payout, `/payouts/*`, read API untuk Dana Saya (dipakai A-017)
- **Dependency:** B-011, Kontrak C9 (MockRentalReadPort) • **Can Start When:** B-011 selesai
- **Deliverable:** TERJADWAL (H+1) → DIPROSES → BERHASIL/GAGAL → retry; nominal = sewa – komisi (komisi TBD → parameter); referensi bank tercatat; rekap bulanan unduhan
- **Acceptance:** payout gagal → retry terjadwal + notifikasi alasan; escrow → DITERUSKAN hanya setelah BERHASIL • **Estimasi:** 2.5 hari

B-015 — Cash-Record Fallback (FR-PAYMENT-009)

- **Domain:** Payment (BR-035/037) • **Input:** B-004 • **Output:** pencatatan cash di `/payments/*`
- **Dependency:** B-004 • **Can Start When:** Sprint 4 (kecil; bisa diselipkan)
- **Deliverable:** record CASH (`is_escrow=false`); masuk GMV + ledger berlabel; komisi → saldo tagihan; TIDAK membuka kunci handover (default, Business Decision Required untuk mengubah)
- **Acceptance:** cash tercatat tidak mengubah EscrowState; proteksi escrow tidak berlaku untuk porsi cash • **Estimasi:** 1.5 hari

B-016 — Financial Admin: Refund Approval & Payout Oversight

- **Domain:** Admin (FR-ADMIN-005–006, SEC-002) • **Input:** B-007, B-014 • **Output:** `/admin/finance/*`
- **Dependency:** B-007, B-014, A-002 (RBAC) • **Can Start When:** keduanya selesai
- **Deliverable:** approval refund di atas ambang TBD (dua tahap, approver ≠ pengaju); oversight payout; semua aksi diaudit dengan identitas individu
- **Acceptance:** pengaju = penyetuju untuk refund besar → ditolak (SEC-002) • **Estimasi:** 2 hari

12. Sprint Plan

6 sprint × 2 minggu. Setiap sprint diakhiri integration checkpoint (Bagian 13). Estimasi total per dev per sprint ≈ 8–10 hari kerja (sisanya untuk review, contract test, buffer).

Sprint 0 — Foundation & Contract (minggu 1–2)

	Developer A	Developer B	Bersama
Task	A-001 (auth), A-002 (RBAC), A-003 (user)	B-001 (devops), B-002 (notif infra), B-003 (audit infra)	Tulis kontrak C1–C9 + mock di <code>app/contracts/</code> ; bekukan signature
Integration point	—	—	Checkpoint 1: semua contract test hijau (mock vs mock); CI jalan

Aturan Sprint 0: tidak boleh ada kode domain sebelum kontrak dibekukan. Kontrak yang berubah setelah Sprint 0 wajib lewat proses `contract-change` (PR label + persetujuan keduanya).

Sprint 1 — Core Domain (minggu 3–4)

	Developer A	Developer B
Task	A-004 (rental), A-005 (verif. rental), A-006 (vehicle), A-007 (availability)	B-004 (payment init), B-005 (webhook), B-006 (escrow+ledger)

	Developer A	Developer B
Bekerja dengan	—	MockBookingPort (C2) — tanpa menunggu A
Integration point	Checkpoint 2: A: rental → vehicle → availability e2e (internal). B: payment → webhook → escrow e2e dengan mock booking + MockPG	

Sprint 2 — Marketplace + Transaction Start (minggu 5–6)

	Developer A	Developer B
Task	A-008 (listing), A-009 (search), A-010 (compare), A-011 (booking + link DP)	B-007 (refund), B-008 (e-KYC), B-009 (agreement)
Bekerja dengan	MockPaymentReadPort (C1) — tanpa menunggu B	MockBookingPort (C2)
Integration point	Checkpoint 3: booking → DP (mock PG) → webhook → booking state berubah. Pertama kali A & B tersambung — via kontrak, bukan via kode.	

Sprint 3 — Trust & Operations (minggu 7–8)

	Developer A	Developer B
Task	A-012 (confirm/cancel), A-013 (extend), A-014 (dashboard), A-015 (offline orders)	B-010 (handover gate), B-011 (return), B-012 (deposit/claim)
Bekerja dengan	MockRefundPort (C5), MockVerificationPort (C4)	MockBookingPort + MockEscrowReadPort (C2, C3)
Integration point	Checkpoint 4: alur trust penuh dengan mock: booking → verifikasi → konfirmasi → handover (gate LUNAS) → return → deposit window	

Sprint 4 — Money Out & Real Integration (minggu 9–10)

	Developer A	Developer B
Task	A-016 (review), A-017 (Dana Saya), A-018 (admin moderasi)	B-013 (dispute), B-014 (payout), B-015 (cash fallback), B-016 (financial admin)
Bekerja dengan	Mock payout/ledger read (kontrak B)	MockRentalReadPort (C9); PG sandbox & e-KYC sandbox mulai disambung (B)
Integration point	Checkpoint 5: payout H+1 e2e (sandbox); dispute → bekukan → resolusi e2e; Dana Saya tampil	

Sprint 5 — Hardening (minggu 11–12)

	Developer A	Developer B	Bersama
Task	Bugfix marketplace; admin final	Bugfix trust/money; ganti mock → real bertahap	E2E penuh (SRS 16.9); security test (16.8); webhook chaos test; docs API (OpenAPI); readiness pilot
Integration point	Checkpoint 6: seluruh mock dimatikan di staging; E2E bahagia + 3 skenario gagal (batal otomatis, klaim deposit, payout gagal) hijau		

Catatan: B tidak pernah "menunggu A menyelesaikan booking" dan A tidak pernah "menunggu B menyelesaikan payment" — setiap sprint keduanya punya task yang bisa selesai dengan mock. Ketergantungan antarsprint hanya pada **kontrak**, bukan implementasi.

13. Integration Checkpoints

Checkpoint	Kapan	Yang diintegrasikan	Kriteria lolos	Jika gagal
1. Foundation Contract	Akhir Sprint 0	Kontrak C1–C9 + mock; CI; auth	Contract test hijau (mock ↔ mock); docker compose up jalan	Jangan mulai Sprint 1
2. Core Domain	Akhir Sprint 1	A: rental → availability; B: payment → escrow (mock booking)	E2E internal masing-masing hijau	Perbaiki dalam Sprint 2 (jangan tunda)
3. Booking + Payment	Akhir Sprint 2	Booking (real A) ↔ Payment (real B) via kontrak	dp_paid mengubah BookingState; link kedaluwarsa → KEDALUWARSA	Bekukan fitur baru; fokus integrasi
4. Trust Flow	Akhir Sprint 3	Verifikasi → konfirmasi → handover gate → return → deposit	Checklist terkunci sebelum LUNAS (test 423); claim window jalan	—
5. End-to-End Transaction	Akhir Sprint 4	Payout H+1 (sandbox), dispute, Dana Saya	Ledger seimbang di semua skenario; refund ≤ dana ditahan	—
6. Hardening	Akhir Sprint 5	Mock → real; security & chaos test	SRS 16.5–16.9 hijau; tidak ada mock di path production staging	Tunda pilot, bukan kurangi test

Frekuensi merge: merge ke `develop` setiap PR (harian), BUKAN menunggu checkpoint. Checkpoint adalah **verifikasi kontrak**, bukan acara merge besar.

14. Git Workflow

Dirancang untuk 2 orang — sederhana, tanpa GitFlow yang berat.

```
main          - hanya release yang ditag (v0.1-pilot, dst.)
▲
develop       - integrasi harian; selalu hijau (CI)
▲
feature/a-booking-link-dp      (A)
feature/b-payment-webhook     (B)
fix/a-search-sort
```

Aturan

Area	Aturan
Branch	<code>feature/<owner>-<domain>-<slug></code> ; owner = <code>a</code> / <code>b</code> . Contoh: <code>feature/a-listing-freshness</code> , <code>feature/b-payout-retry</code> .
Commit	Conventional Commits: <code>feat(bookings): ...</code> , <code>fix(payments): ...</code> , <code>test(ledger): ...</code> , <code>docs: ...</code> , <code>chore: ...</code> . Scope = nama domain.
PR	Template wajib: ringkasan, kontrak yang disentuh (ya/tidak), migration (ya/tidak + arah), test yang dijalankan, checklist DoD.
Review	1 reviewer = developer satunya. Review fokus: (1) tidak ada import lintas domain; (2) kontrak tidak berubah diam-diam; (3) aturan uang tidak dilonggarkan.
Merge	Squash merge ke <code>develop</code> . Merge hanya bila CI hijau.
Konflik	Konflik kode: selesaikan di branch feature. Konflik migration: lihat Bagian 15 (tidak pernah edit migration yang sudah di- <code>develop</code>).

Area	Aturan
Release	Dari <code>develop</code> yang hijau → tag <code>vx.Y</code> di <code>main</code> . Pilot = <code>v0.1-pilot</code> .

Larangan: push langsung ke `develop` / `main`; merge PR sendiri; menggabungkan dua domain besar dalam satu PR (maksimal 1 domain per PR agar review efektif).

15. Migration Strategy (Alembic)

15.1 Keputusan: satu migration history, konvensi nama ketat

Mengapa bukan prefix A001/B001 terpisah? Alembic mengurutkan migration berdasarkan rantai `down_revision`, BUKAN nama file. Dua sekuens prefix terpisah dalam satu history akan bertabrakan saat digabung (dua head). Solusi yang benar:

- Satu history linear.** Setiap migration punya tepat satu `down_revision` = head terakhir di `develop`.
- Konvensi nama:** `YYYYMMDDHHMMSS_<owner>_<domain>_<deskripsi>.py` — contoh:
`20261004120000_a_rentals_add_payout_account.py`. Owner (`a` / `b` / `shared`) untuk akuntabilitas, BUKAN untuk ordering.
- Satu migration = satu owner domain** (kecuali baseline). Migration B dilarang menyentuh tabel A.

15.2 Workflow anti-konflik

- ```
1. git pull develop terbaru
2. alembic revision --autogenerate -m "<timestamp>_<owner>_<domain>_<desc>"
3. Periksa hasil autogenerate MANUAL (autogenerate tidak sempurna):
 - pastikan hanya tabel milikmu yang berubah
 - pastikan tidak ada DROP yang tidak disengaja
 - isi data migration bila perlu (mis. backfill) — jangan di autogenerate
4. alembic upgrade head → pytest → alembic downgrade -1 → alembic upgrade head
5. PR dengan label migration; reviewer memeriksa file migration
```

**Jika dua migration dibuat dari head yang sama** (konflik): developer yang merge belakangan **wajib** meregenerasi migration-nya (`down_revision` = migration yang duluan merge). Tidak ada "pilih salah satu".

**Larangan keras:** mengedit migration yang sudah merge ke `develop`; menghapus migration; `alembic stamp` manual tanpa persetujuan.

### 15.3 CI migration check

Setiap PR menjalankan: `alembic upgrade head` pada database kosong → `downgrade -1` → `upgrade head` → verifikasi tidak ada multiple heads (`alembic heads` = tepat 1). Gagal = PR tidak bisa merge.

## 16. Testing Strategy

### 16.1 Piramida test

```

 /\
 /E2E\ sedikit: 8 skenario SRS 16.9 (bahagia + gagal kritis)
 /-----\
/Contract\ 1 suite per kontrak × (mock + real)
/-----\
/Integration \ API + DB per domain; webhook dengan fixture
/-----\
/ Unit \ banyak: state machine, kalkulasi, aturan bisnis
/-----\
```

## 16.2 Layout & konvensi

```
tests/
├─ unit/domains/<domain>/test_<aturan>.py
├─ contract/test_c1_payment_status.py # parametrize: mock & real
├─ integration/test_bookings_api.py
├─ integration/test_webhook_pg.py # fixture: tests/fixtures/pg_callback_*.json
└─ e2e/test_happy_path.py # booking-payout penuh
```

- **Unit:** logika murni (state machine booking/escrow, hitung refund tier, claim window). Cepat, tanpa DB bila bisa.
- **Contract:** suite yang sama dijalankan ke mock dan implementasi real. Invarian uang (ledger seimbang, refund  $\leq$  ditahan) diuji di sini.
- **Integration:** API + Postgres asli (test DB). Auth selalu real (JWT asli).
- **E2E:** 8 skenario SRS 16.9, dijalankan di CI malam hari (boleh lambat), wajib hijau sebelum release.

## 16.3 Fixture & data

- Fixture PG callback: `tests/fixtures/pg/` — sukses, gagal, kedaluwarsa, duplikat, signature salah, terlambat-setelah-kedaluwarsa.
- Fixture e-KYC: KTP valid, meragukan, palsu.
- Factory: `booking_factory`, `rental_factory` — milik A, boleh dipakai test B via import eksplisit (test boleh lintas domain; kode production tidak).

## 16.4 Target

- Coverage:  $\geq 80\%$  untuk domain B (uang),  $\geq 70\%$  untuk domain A. Angka di bawah ini = PR ditahan.
- Setiap bug yang ditemukan di staging/prod → wajib ada regression test sebelum fix di-merge.

# 17. Definition of Done

Sebuah task backend dinyatakan **Done** jika dan hanya jika SEMUA di bawah terpenuhi:

- Implementasi selesai sesuai FR/BR SRS (tidak lebih, tidak kurang — scope creep = PR baru)
- Validasi input lengkap (tipe, format, rentang) di sisi server
- Authorization diperiksa (role + kepemilikan); IDOR test ada bila relevan
- Unit test hijau (aturan bisnis & state machine ter-cover)
- Integration/contract test hijau bila menyentuh kontrak atau DB
- Error handling: setiap kegagalan mengembalikan kode error baku + `trace_id`; tidak ada 500 telanjang untuk error bisnis
- Logging: event penting tercatat; tidak ada data sensitif di log (SEC-005/007)
- Migration (bila ada): mengikuti Bagian 15; `upgrade/downgrade/upgrade` hijau
- OpenAPI terdokumentasi otomatis (docstring + schema jelas); contoh request/response untuk endpoint baru
- Lint (ruff) & typecheck (mypy) bersih
- PR direview developer satunya + CI hijau + squash merge ke `develop`
- Untuk domain uang (B): invarian **ledger seimbang** dan **refund  $\leq$  dana ditahan** dibuktikan oleh test, bukan oleh janji

# 18. Critical Path

## 18.1 Critical path (jalur yang menentukan tanggal selesai MVP)

```
Auth/RBAC (A-001/002, Sp0)
└─ User (A-003, Sp0)
 └─ Rental (A-004/005, Sp1)
 └─ Vehicle (A-006, Sp1)
```

- Availability (A-007, Sp1)
- Listing (A-008, Sp2)
- Booking (A-011, Sp2)
  - Payment+Escrow (B-004/005/006, Sp1) → paralel via kontrak C1/C2
  - Handover gate (B-010, Sp3)
  - Return (B-011, Sp3)
  - Deposit/Payout (B-012/014, Sp3-4)

**Keterlambatan di jalur ini = keterlambatan MVP.** Dua titik paling berisiko: **A-011** (booking adalah simpul tersibuk) dan **B-005/B-006** (webhook + escrow adalah invariansi tersulit).

## 18.2 Parallel paths (bisa jalan kapan saja tanpa memblokir critical path)

| Path               | Task                | Keterangan                                 |
|--------------------|---------------------|--------------------------------------------|
| Search & discovery | A-009, A-010        | Butuh listing fixture; tidak butuh payment |
| Trust dokumen      | B-008, B-009        | e-KYC + agreement; mock booking cukup      |
| Operasional rental | A-014, A-015, A-018 | Dashboard; tidak menyentuh uang            |
| Reputasi           | A-016               | Butuh kontrak status booking saja          |
| Sengketa           | B-013               | Butuh handover/return (internal B)         |
| Infra              | B-001/002/003       | Sprint 0, lalu selesai                     |

## 18.3 Blocking vs non-blocking

| Task                        | Blocking?                            | Alasan                                                                                                                                                                                                    |
|-----------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A-001/002 (auth/RBAC)       | <b>BLOCKING</b>                      | Semua endpoint butuh ini. Mitigasi: selesai Sprint 0 (prioritas tertinggi).                                                                                                                               |
| Kontrak C1–C9               | <b>BLOCKING</b>                      | Tanpa ini paralelisme runtuh. Mitigasi: ditulis berdua di Sprint 0, dibekukan.                                                                                                                            |
| B-004/005/006               | <b>BLOCKING</b> untuk integrasi uang | Tapi BISA dibangun dengan mock booking — blocking hanya untuk tahap integrasi real, bukan untuk development.                                                                                              |
| A-011                       | <b>BLOCKING</b> untuk A-012/013/014  | Internal A; dipecah kecil agar tidak macet.                                                                                                                                                               |
| A-009, B-008, A-014, B-013  | Non-blocking                         | Punya mock/kontrak yang cukup; bisa digeser antarsprint.                                                                                                                                                  |
| Provider real (PG/e-KYC/WA) | Non-blocking untuk dev               | Mock menutupinya sampai Sprint 4. <b>Satu-satunya</b> dependensi eksternal yang benar-benar blocking adalah <b>keputusan pemilihan provider</b> (OQ-024–026) — tanpa itu tidak bisa sign kontrak sandbox. |

# 19. Parallel Work Opportunities

Pasangan task yang **sengaja** dirancang bisa dikerjakan bersamaan (inilah inti plan ini):

| Sprint | Developer A                     | Developer B                 | Penghubung                                                |
|--------|---------------------------------|-----------------------------|-----------------------------------------------------------|
| 0      | Auth/RBAC/User                  | DevOps/Notif/Audit infra    | Kontrak C1–C9 ditulis berdua                              |
| 1      | Rental → Vehicle → Availability | Payment → Webhook → Escrow  | MockBookingPort ↔ (tanpa menunggu)                        |
| 2      | Listing → Search → Booking      | Refund, e-KYC, Agreement    | MockPaymentReadPort ↔ MockBookingPort                     |
| 3      | Confirm/Cancel/Dashboard        | Handover → Return → Deposit | MockVerificationPort, MockRefundPort ↔ MockEscrowReadPort |

| Sprint | Developer A              | Developer B                    | Penghubung                                      |
|--------|--------------------------|--------------------------------|-------------------------------------------------|
| 4      | Review, Dana Saya, Admin | Dispute, Payout, Cash fallback | MockRentalReadPort ↔ kontrak read payout/ledger |
| 5      | Bugfix + docs            | Bugfix + mock → real           | E2E bersama                                     |

**Aturan main:** jika A selesai lebih cepat di suatu sprint, A mengambil (prioritas): (1) contract test tambahan, (2) membantu review PR B, (3) task non-blocking dari sprint berikut — **bukan** mengerjakan domain B. Begitu pula sebaliknya. Kepemilikan domain tidak boleh bocor.

## 20. Blocking Risks

| Risiko                                                        | Probability | Impact        | Mitigation                                                                                                                       | Owner       |
|---------------------------------------------------------------|-------------|---------------|----------------------------------------------------------------------------------------------------------------------------------|-------------|
| Schema conflict (dua migration menyentuh tabel sama)          | Medium      | High          | Aturan 5.3: satu tabel = satu owner; CI cek single head; regenerasi bila konflik                                                 | Keduanya    |
| API contract mismatch (implementasi menyimpang dari kontrak)  | Medium      | High          | Contract test mock-vs-real di CI; perubahan kontrak wajib label <code>contract-change</code> + persetujuan                       | Keduanya    |
| Payment webhook issue (duplikat, terlambat, signature salah)  | Medium      | Sangat tinggi | SEC-011/012; fixture lengkap; chaos test di Sprint 5                                                                             | B           |
| State machine inconsistency (booking vs escrow tidak selaras) | Low         | Sangat tinggi | Monitor inkonsistensi (SRS 15.2); transisi hanya via Port; unit test semua transisi terlarang                                    | A+B         |
| Migration conflict (dua head)                                 | Medium      | Medium        | Workflow 15.2; CI <code>alembic heads = 1</code>                                                                                 | Keduanya    |
| External provider unavailable (PG/e-KYC/WA belum dipilih)     | Medium      | High          | Mock menutupi dev sampai Sprint 4; <b>keputusan provider (OQ-024-026) dikejar paralel sejak Sprint 1</b>                         | B + Product |
| Developer waiting (satu pihak telat menyelesaikan kontrak)    | Low         | High          | Kontrak dibekukan Sprint 0; mock memungkinkan kerja mandiri; checkpoint tiap 2 minggu                                            | Keduanya    |
| Integration bug (lolos unit, gagal saat disambung)            | Medium      | Medium        | Checkpoint 3–5; E2E dengan invariansi uang; tidak ada "big bang integration"                                                     | Keduanya    |
| Unclear ownership (abu-abu siapa mengerjakan apa)             | Low         | Medium        | Bagian 4–5 sebagai hukum; sengketa ownership diputus: "yang paling dekat dengan uang = B, yang paling dekat dengan tampilan = A" | Keduanya    |
| Scope creep (fitur future menyusup ke MVP)                    | Medium      | Medium        | SRS 17 sebagai pagar; PR yang menyentuh future scope ditolak dengan rujukan                                                      | Keduanya    |
| Estimasi meleset (domain B lebih dalam dari perkiraan)        | Medium      | Medium        | B diberi domain lebih sedikit; buffer di Sprint 5; task B-013 bisa digeser tanpa memblokir payout                                | B           |

## 21. Final Recommended Development Order

Urutan pembangunan yang direkomendasikan — dibaca sebagai checklist eksekusi:

- Sprint 0:** Bekukan kontrak C1–C9 + mock (berdua). A: auth/RBAC/user. B: docker/alembic/pytest/CI + notification/audit infra.
- Sprint 1:** A: rental → vehicle → availability (tulang punggung supply). B: payment init → webhook → escrow + ledger (tulang punggung uang), melawan mock booking.

3. **Sprint 2:** A: listing → search → booking + link DP (melawan mock payment). B: refund engine, e-KYC, agreement. **Checkpoint 3:** sambungkan booking ↔ payment via kontrak — integrasi nyata pertama.
4. **Sprint 3:** A: konfirmasi/pembatalan/perpanjangan + dashboard + offline orders. B: handover gate (kunci sampai LUNAS) → return → deposit/claim window.
5. **Sprint 4:** A: review + Dana Saya (baca via kontrak) + admin moderasi. B: dispute + payout H+1 + cash fallback + financial admin. Sambungkan PG/e-KYC sandbox (keputusan provider harus sudah ada).
6. **Sprint 5:** Matikan mock bertahap di staging. E2E penuh + security test + webhook chaos test. Dokumentasi OpenAPI final. **Definisi "siap pilot":** Checkpoint 6 hijau.

**Tiga hal yang tidak boleh ditawarkan:**

1. **Kontrak dulu, kode kemudian** — setiap ketergantungan lintas developer yang belum punya kontrak adalah BLOCKING DEPENDENCY yang harus diubah menjadi CONTRACT DEPENDENCY sebelum Sprint 1.
2. **Uang tidak pernah menunggu tampilan, tampilan tidak pernah menunggu uang** — A dan B masing-masing punya jalur kerja mandiri penuh di setiap sprint via mock.
3. **Satu-satunya blokir eksternal yang nyata adalah pemilihan provider** (payment gateway, e-KYC, notifikasi — OQ-024-026). Kejar keputusannya sejak Sprint 1, paralel dengan development, agar Sprint 4 tidak tersandera.

---

*Dokumen ini adalah pedoman kerja, bukan birokrasi. Jika sebuah aturan di sini memperlambat tanpa menambah keamanan/kualitas, diskusikan berdua dan ubah aturannya — tetapi catat perubahannya.*